

Heaven's light is our guide"

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

Compiler Design

Course No. : CSE 701

Chapter 4: Syntax Analysis

Prepared By : Julia Rahman

Overview

Why are Grammars to formally describe Languages Important ?

1. Precise, easy-to-understand representations
 2. Compiler-writing tools can take grammar and generate a compiler
 3. Allow language to be evolved (new statements, changes to statements, etc.)
- Languages are not static, but are constantly upgraded to add new features or fix “old” ones

Why study lexical and syntax analyzers?

- ✓ Lexical and syntax analysis not just used in compiler design:
 - program listing formatters
 - programs that compute complexity
 - programs that analyze and react to configuration files

4.1 The Role of the Parser

- Syntax analysis is done by the parser.

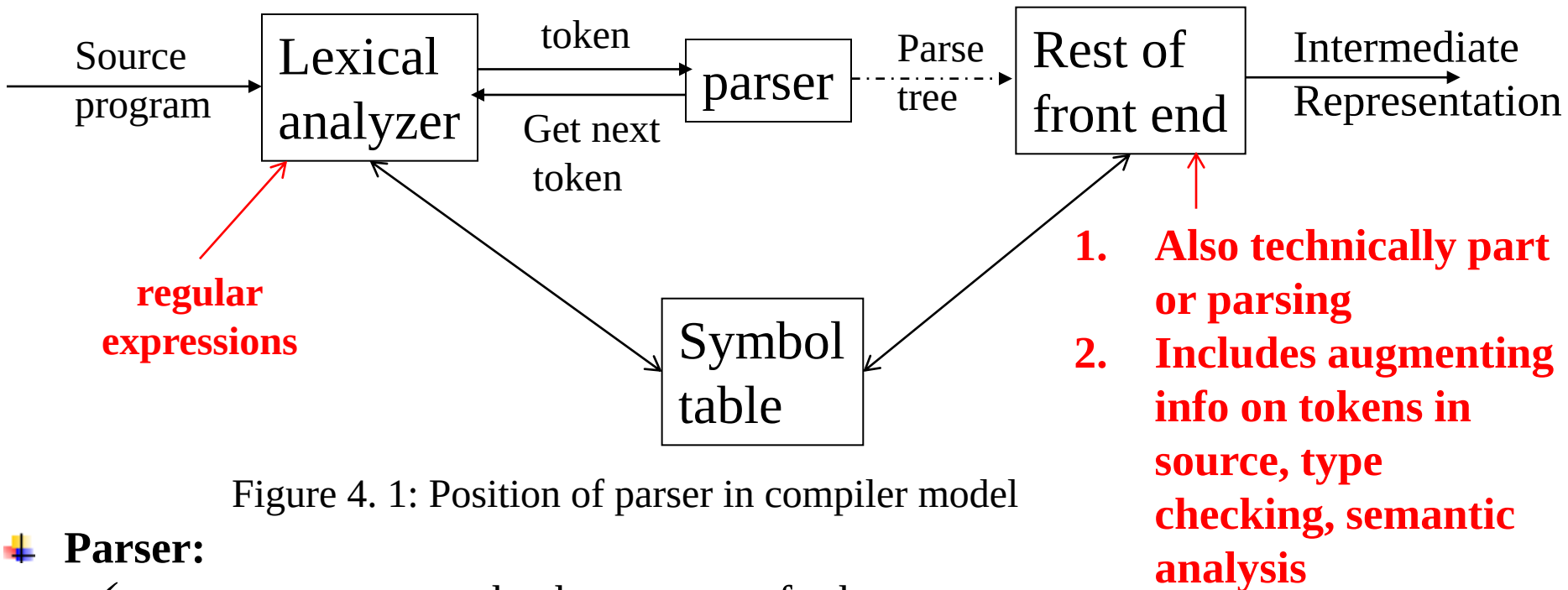


Figure 4. 1: Position of parser in compiler model



Parser:

- ✓ uses a grammar to check structure of tokens
- ✓ produces a parse tree
- ✓ syntactic errors and recovery
- ✓ recognize correct syntax
- ✓ report errors

4.1 The Role of the Parser

Syntax Error Handling:

1) Lexical errors:

- ✓ Include misspellings of identifiers, keywords, or operators
- ✓ Example: the use of an identifier `elipseSize` instead of `ellipseSize` – and missing quotes around text intended as a string.

2) Syntactic errors:

- ✓ Omission, wrong order of tokens
- ✓ include misplaced semicolons or extra or missing braces; that is, "{" or "}".

3) Semantic errors:

- ✓ Incompatible types
- ✓ include type mismatches between operators and operands.
- ✓ An example is a return statement in a Java method with result type `void`.

4) Logical errors:

- ✓ anything from incorrect reasoning on the part of the programming.
- ✓ Infinite loop / recursive call

 Majority of error processing occurs during syntax analysis.

 **NOTE: Not all errors are identifiable.**

4.1 The Role of the Parser

+ Error handler goals

- ✓ Report the presence of errors clearly and accurately
- ✓ Recover from each error quickly enough to detect subsequent errors
- ✓ Add minimal overhead to the processing of correct programs

+ Error-recover strategies:

1) Panic mode recovery

- ✓ Discard input symbol one at a time until one of designated set of synchronization tokens is found
- ✓ The synchronizing tokens are usually delimiters, such as semicolon or }
- ✓ **Advantages:**
 - simple $\Rightarrow$ suited to 1 error per statement
- ✓ **Problems:**
 - skip input $\Rightarrow$ miss declaration – causing more errors
 - $\Rightarrow$ miss errors in skipped material

2) Phrase level recovery

- ✓ Replacing a prefix of remaining input by some string that allows the parser to continue
- ✓ Local correction on input is to replace a comma by a semicolon, delete an extraneous semicolon, or insert a missing semicolon.
- ✓ Not suited to all situations
- ✓ Used in conjunction with panic mode to allow less input to be skipped

4.1 The Role of the Parser

3) **Error productions**

- ✓ Augment the grammar with productions that generate the erroneous constructs
- ✓ Example: add a rule for
 `:=` in C assignment statements
 Report error but continue compile
- ✓ Self correction + diagnostic messages

4) **Global correction**

- ✓ Choosing minimal sequence of changes to obtain a globally least-cost correction
- ✓ Adding / deleting / replacing symbols
- ✓ Costly - key issues

4.2 Context Free Grammars



Context Free Grammars:

- ✓ Basis of parsing
- ✓ Represent language constructs
- ✓ consists of terminals, nonterminals, a start symbol, and productions.

stmt $\rightarrow$ if (expr) stmt else stmt

1) **Terminals :**

- ✓ tokens of the language
- ✓ the terminals are the keywords if and else and the symbols " c" and ") ."

2) **Non-terminals:**

- ✓ denote sets of strings generated by the grammar & in the language
- ✓ stmt and expr are nonterminals

3) **Start symbol:**

- ✓ one nonterminal is distinguished as the start symbol,
- ✓ the productions for the start symbol are listed first.

4) **Production rules:**

- ✓ to indicate how T and NT are combined to generate valid strings of the language.

4.2 Context Free Grammars

- ✓ Each production consists of:
 - (a) A **nonterminal** called the head or left side of the production; this production defines some of the strings denoted by the head.
 - (b) The **symbol** $\rightarrow$ Sometimes $::=$ has been used in place of the arrow.
 - (c) A **body or right side** consisting of zero or more terminals and nonterminals. The components of the body describe one way in which strings of the nonterminal at the head can be constructed.

 **Example 4.2 :** The grammar with the following production defines simple arithmetic expressions. In this grammar, the terminal symbols are

$\text{id} + - * / \uparrow ()$

Production: $\text{expression} \rightarrow \text{expression op expression}$

$\text{expression} \rightarrow (\text{expression})$

$\text{expression} \rightarrow - \text{expression}$

$\text{expression} \rightarrow \text{id}$

$\text{op} \rightarrow +$

$\text{op} \rightarrow -$

$\text{op} \rightarrow *$

$\text{op} \rightarrow /$

$\text{op} \rightarrow \uparrow$

4.2 Context Free Grammars

✚ Notational Convent:

Terminals: a,b,c,+,-,punc,0,1,...,9

Non Terminals: A,B,C,S

T or NT: X,Y,Z

Strings of Terminals: u,v,...,z in T^*

Strings of T / NT: α, β, γ in $(T \cup NT)^*$

Alternatives of production rules:

$A \rightarrow \alpha_1; A \rightarrow \alpha_2; \dots; A \rightarrow \alpha_k; \Rightarrow A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$

First NT on LHS of 1st production rule is designated as start symbol !

✚ Example 4.3:

Using these shorthands, the grammar of Example 4.2 can be rewritten concisely as

$$E \rightarrow E A E \mid (E) \mid -E \mid id$$
$$A \rightarrow + \mid - \mid * \mid / \mid \uparrow$$

The notational conventions tell us that E, and A are nonterminals, with E the start symbol. The remaining symbols are terminals.

4.2 Context Free Grammars

Derivations:

- ✓ A step in a derivation is zero or one action that replaces a NT with the RHS of a production rule.
- ✓ Productions are treated as rewriting rules to generate a string

EXAMPLE: $E \Rightarrow -E$ (the $\Rightarrow$ means “derives” in one step) using the production rule: $E \rightarrow -E$

EXAMPLE: $E \Rightarrow E A E \Rightarrow E * E \Rightarrow E * (E)$

DEFINITION: $\Rightarrow$ derives in one step
 $\Rightarrow$ derives in $\geq$ one step
 $\Rightarrow$ derives in $\geq$ zero steps

Leftmost: Replace the leftmost non-terminal symbol

$E \Rightarrow E A E \Rightarrow \text{id} A E \Rightarrow \text{id} * E \Rightarrow \text{id} * \text{id}$

Rightmost: Replace the leftmost non-terminal symbol

$E \Rightarrow E A E \Rightarrow E A \text{id} \Rightarrow E * \text{id} \Rightarrow \text{id} * \text{id}$

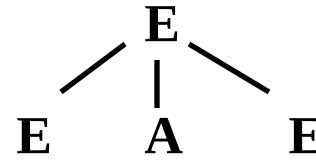
Example 4.4: The string $-(\text{id}+\text{id})$ is a sentence of grammar because there is the derivation

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(\text{id}+E) \Rightarrow -(\text{id}+\text{id})$

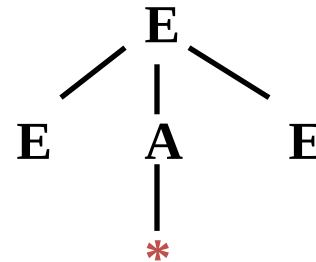
4.2 Context Free Grammars

Derivations & Parse Tree:

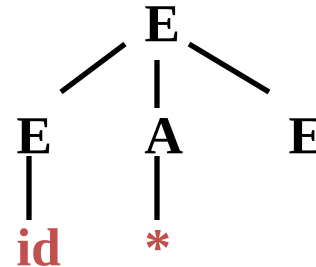
$E \Rightarrow E A E$



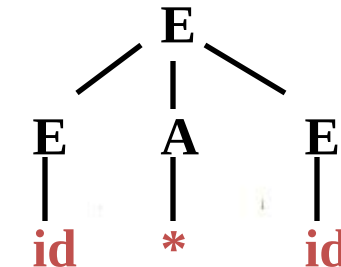
$\Rightarrow E * E$



$\Rightarrow id * E$



$\Rightarrow id * id$

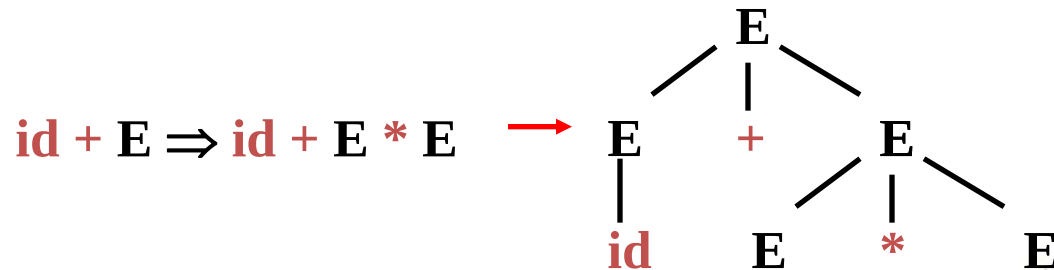
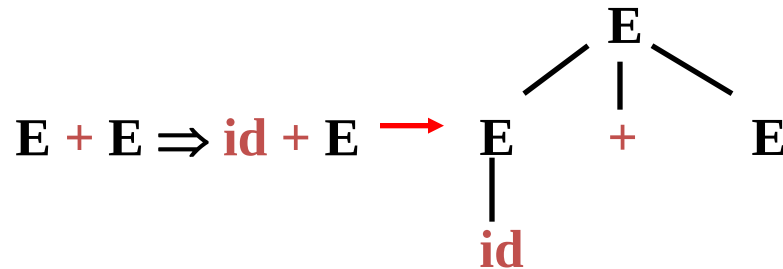
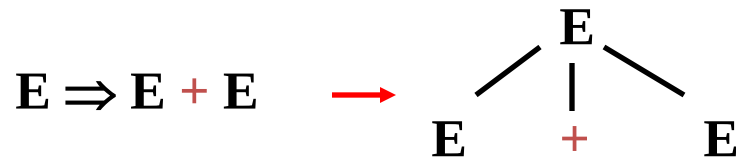


4.2 Context Free Grammars

✚ Consider the expression grammar:

$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$$

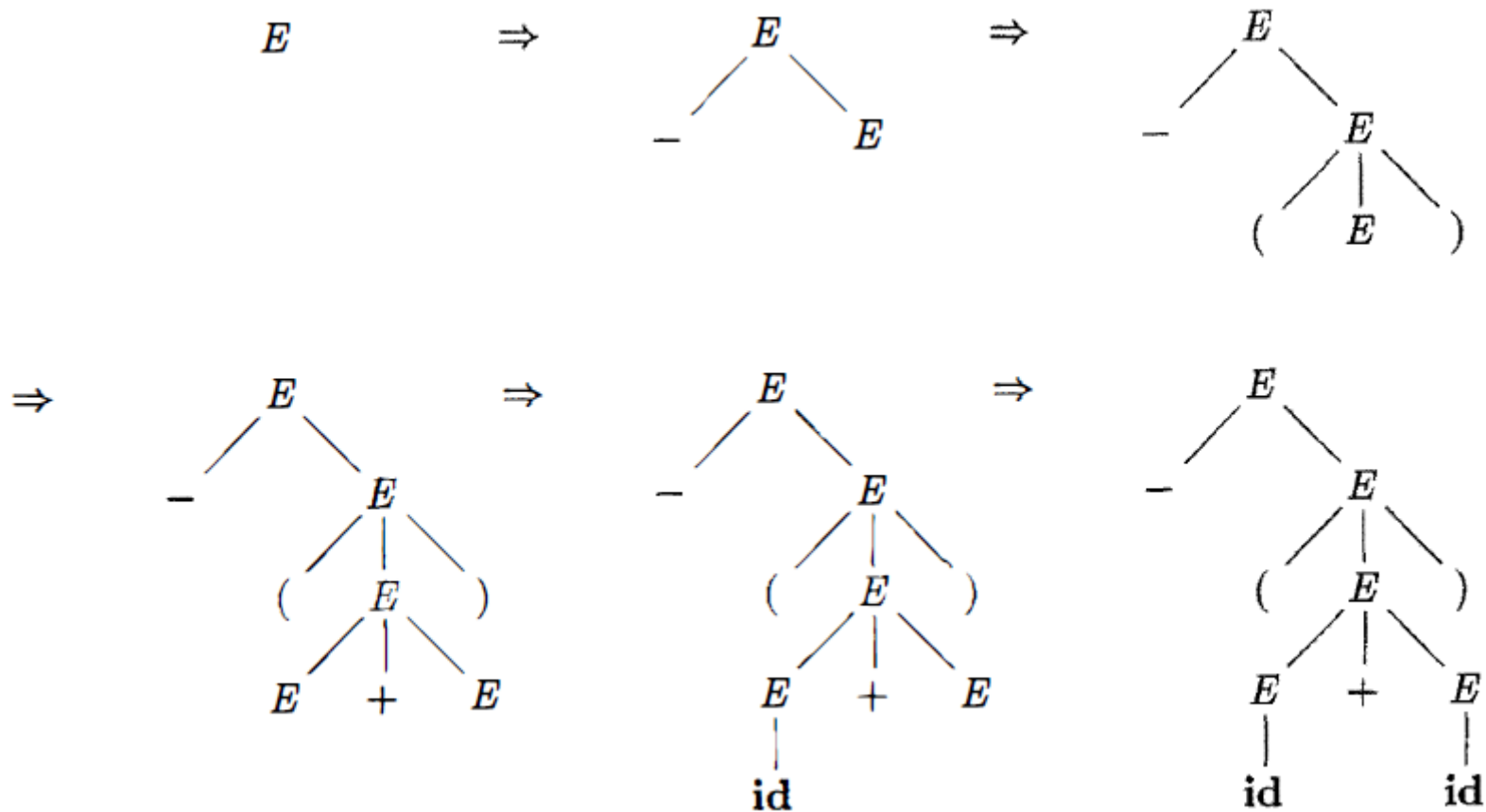
Leftmost derivations of $\text{id} + \text{id} * \text{id}$



4.2 Context Free Grammars

Example 4.5:

Build a parse tree for $-(\text{id}+\text{id})$ or $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(\text{id}+E) \Rightarrow -(\text{id}+\text{id})$



4.2 Context Free Grammars

Example 4.6 :

The arithmetic expression grammar permits two distinct leftmost derivations for the sentence $\text{id} + \text{id} * \text{id}$:

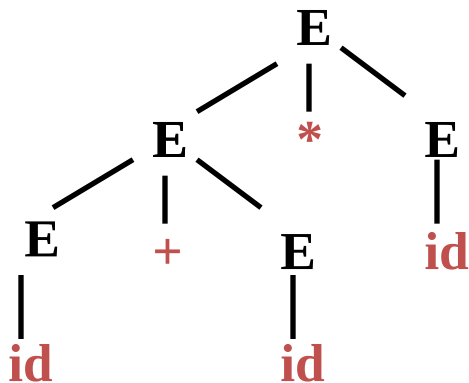
$$E \Rightarrow E * E$$

$$\Rightarrow E + E * E$$

$$\Rightarrow \text{id} + E * E$$

$$\Rightarrow \text{id} + \text{id} * E$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$



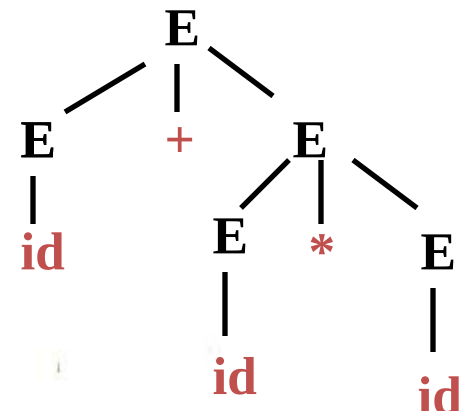
$$E \Rightarrow E + E$$

$$\Rightarrow \text{id} + E$$

$$\Rightarrow \text{id} + E * E$$

$$\Rightarrow \text{id} + \text{id} * E$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

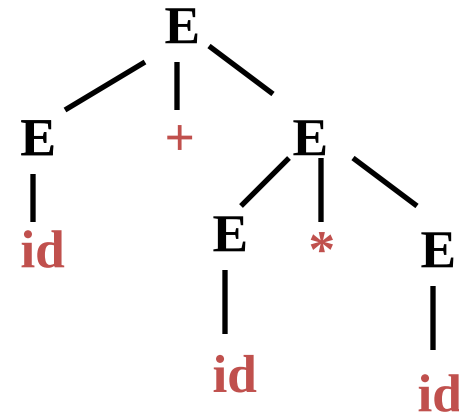
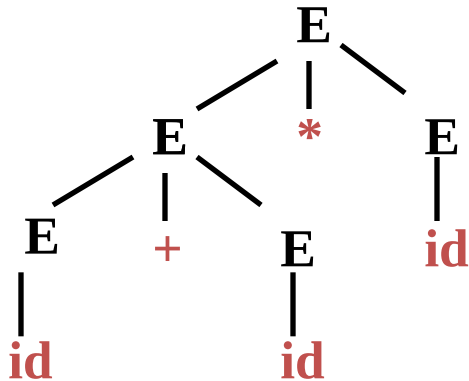


4.2 Context Free Grammars



Ambiguity:

- ✓ For some strings there exist more than one parse tree
- ✓ Or more than one leftmost derivation
- ✓ Or more than one rightmost derivation
- ✓ Example: $\text{id} + \text{id} * \text{id}$



4.3 Writing a Grammar



"Why use regular expressions to define the lexical syntax of a language?"

Answer: There are several reasons:

- 1 . Separating the syntactic structure of a language into lexical and nonlexical parts provides a convenient way of modularizing the front end of a compiler into two manageable-sized components.
2. The lexical rules of a language are frequently quite simple, and to describe them we do not need a notation as powerful as grammars.
3. Regular expressions generally provide a more concise and easier-to-understand notation for tokens than grammars.
4. More efficient lexical analyzers can be constructed automatically from regular expressions than from arbitrary grammars.

4.3 Writing a Grammar

✚ Elimination of left recursion:

- ✓ A grammar is left recursive if it has a non-terminal A such that there is a derivation $A \rightarrow A\alpha$
- ✓ Top down parsing methods cant handle left-recursive grammars
- ✓ A simple rule for direct left recursion elimination:

For a rule like:

$$A \rightarrow A\alpha|\beta$$

We may replace it with

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

✚ **Example 4.8 :** Consider the following grammar for arithmetic expression:

$$E \rightarrow E + T \mid T$$

$$E \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Eliminating the immediate left recursion to the production for E and then T , we obtain

$$E \rightarrow T E'$$

$$E' \rightarrow + T E'$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T'$$

$$F \rightarrow (E) \mid \text{id}$$

4.3 Writing a Grammar

Left factoring:

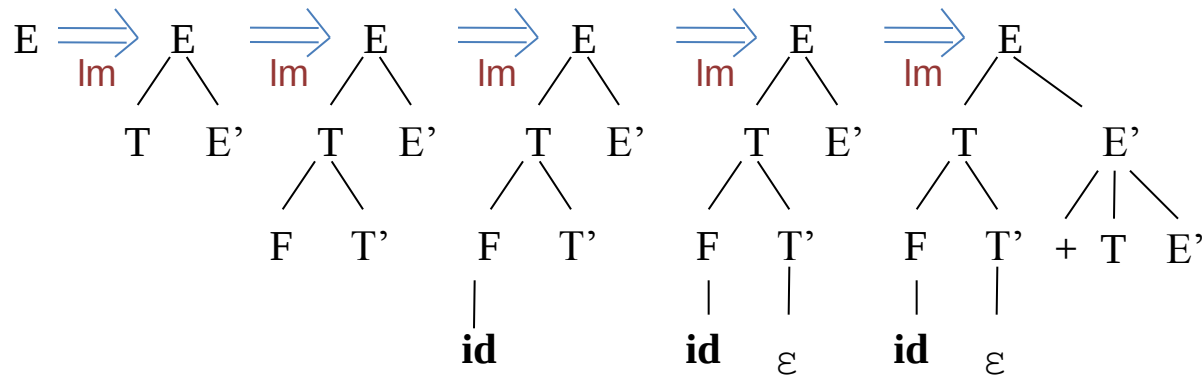
- ✓ Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive or top-down parsing.
- ✓ Consider following grammar:
 - $\text{Stmt} \rightarrow \text{if expr then stmt else stmt} \mid \text{if expr then stmt}$
- ✓ On seeing input **if** it is not clear for the parser which production to use
- ✓ We can easily perform left factoring:
 - If we have $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ then we replace it with
 - $A \rightarrow \alpha A'$
 - $A' \rightarrow \beta_1 \mid \beta_2$
- ✓ Algorithm
 - For each non-terminal A , find the longest prefix α common to two or more of its alternatives. If $\alpha \neq \epsilon$, then replace all of A -productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$ by
 - $A \rightarrow \alpha A' \mid \gamma$
 - $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$
- ✓ Example: $S \rightarrow i E t S \mid i E t S e S \mid a$ and $E \rightarrow b$; here i, t, e stands for **if, then** and **else**, E and S for “**expression**” and “**statement**”. Left factored the grammar $S \rightarrow i E t S S' \mid a$, $S' \rightarrow e S \mid \epsilon$ and $E \rightarrow b$

4.4 Top-Down Parsing



Top-Down Parsing:

- ✓ A Top-down parser tries to create a parse tree from the root towards the leafs scanning input from left to right
- ✓ It can be also viewed as finding a leftmost derivation for an input string
- ✓ Example: `id+id*id`

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \mathbf{id} \end{aligned}$$


4.4 Top-Down Parsing



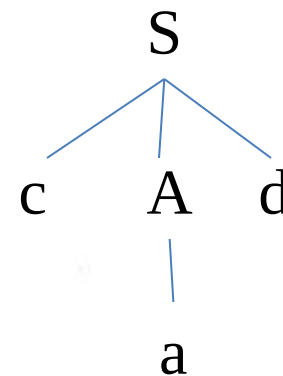
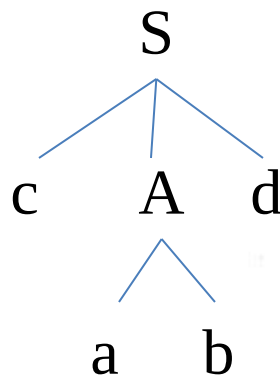
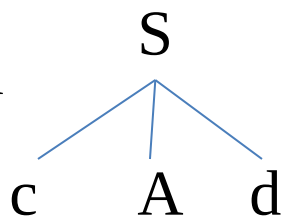
Recursive descent parsing:

- ✓ Consists of a set of procedures, one for each nonterminal
- ✓ Execution begins with the procedure for start symbol
- ✓ A typical procedure for a non-terminal
- ✓ General recursive descent may require backtracking
- ✓ The previous code needs to be modified to allow backtracking
- ✓ In general form it can't choose an A-production easily.
- ✓ So we need to try all alternatives
- ✓ If one failed the input pointer needs to be reset and another alternative should be tried
- ✓ Recursive descent parsers can't be used for left-recursive grammars
- ✓ Example:

Input: cad

$S \rightarrow cAd$

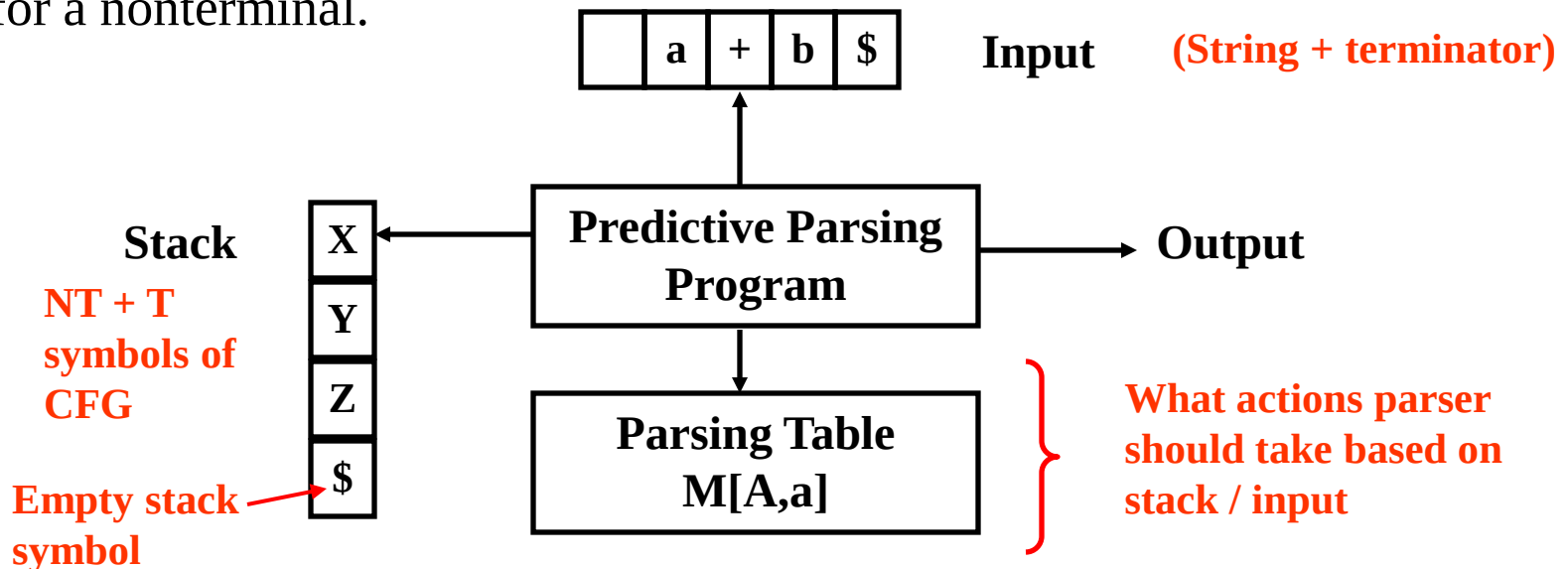
$A \rightarrow ab \mid a$



4.4 Top-Down Parsing

Non-recursive predicting parsing:

- ✓ Key problem of predicting is that of determining the production to be applied for a nonterminal.



❖ **General parser behavior:** X : top of stack a : current input

1. When $X=a=\$$ halt, accept, success
2. When $X=a \neq \$$, POP X off stack, advance input, go to 1.
3. When X is a non-terminal, examine $M[X,a]$
 - if it is an error $\rightarrow$ call recovery routine
 - if $M[X,a] = \{X \rightarrow UVW\}$, POP X , PUSH W,V,U
 - DO NOT expend any input

4.4 Top-Down Parsing

Example:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow + TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow * FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

Non-terminal	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow + TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Figure 4.15: Parsing table for above grammar

4.4 Top-Down Parsing

STACK	INPUT	OUTPUT
\$E	id + id * id\$	
\$E'T	id + id * id\$	$E \rightarrow TE'$
\$E'T'F	id + id * id\$	$T \rightarrow FT'$
\$E'T'id	id + id * id\$	$F \rightarrow id$
\$E'T'	+ id * id\$	
\$E'	+ id * id\$	$T' \rightarrow \epsilon$
\$E'T+	+ id * id\$	$E' \rightarrow \underline{+}TE'$
\$E'T	id * id\$	
\$E'T'F	id * id\$	$T \rightarrow FT'$
\$E'T'id	id * id\$	$F \rightarrow id$
\$E'T'	* id\$	
\$E'T'F*	* id\$	$T' \rightarrow \underline{*}FT'$
\$E'T'F	id\$	
\$E'T'id	id\$	$F \rightarrow id$
\$E'T'	\$	
\$E'	\$	$T' \rightarrow \epsilon$
\$	\$	$E' \rightarrow \epsilon$

Expend Input

Figure 4.16: Moves made by a predictive parser on input id + id * id

4.4 Top-Down Parsing

+ First and Follow:

- ✓ First() is set of terminals that begins strings derived from
- ✓ If $\alpha \rightarrow \epsilon$ then ϵ is also in First(ϵ)
- ✓ In predictive parsing when we have $A \rightarrow \alpha | \beta$, if First(α) and First(β) are disjoint sets then we can select appropriate A-production by looking at the next input
- ✓ Follow(A), for any nonterminal A, is set of terminals a that can appear immediately after A in some sentential form
 - If we have $S \Rightarrow \alpha A a \beta$ for some α and β then a is in Follow(A)
- ✓ If A can be the rightmost symbol in some sentential form, then \$ is in Follow(A)

+ Computing First:

To compute First(X) for all grammar symbols X, apply following rules until no more terminals or ϵ can be added to any First set:

1. If X is a terminal then First(X) = {X}.
2. If X is a nonterminal and $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production for some $k \geq 1$, then place a in First(X) if for some i a is in First(Y_i) and ϵ is in all of First(Y_1), ..., First(Y_{i-1}) that is $Y_1 \dots Y_{i-1} \Rightarrow \epsilon$. if ϵ is in First(Y_j) for $j=1, \dots, k$ then add ϵ to First(X).
3. If $X \rightarrow \epsilon$ is a production then add ϵ to First(X)

4.4 Top-Down Parsing



Computing follow:

To compute $\text{First}(A)$ for all nonterminals A , apply following rules until nothing can be added to any follow set:

1. Place $\$$ in $\text{Follow}(S)$ where S is the start symbol
2. If there is a production $A \rightarrow \alpha B \beta$ then everything in $\text{First}(\beta)$ except ϵ is in $\text{Follow}(B)$.
3. If there is a production $A \rightarrow B$ or a production $A \rightarrow \alpha B \beta$ where $\text{First}(\beta)$ contains ϵ , then everything in $\text{Follow}(A)$ is in $\text{Follow}(B)$



Example:

$S \rightarrow \mathbf{i} E \mathbf{t} S S' \mid \mathbf{a}$	$\text{First}(S) = \{ \mathbf{i}, \mathbf{a} \}$	$\text{Follow}(S) = \{ \mathbf{e}, \$ \}$
$S' \rightarrow \mathbf{e} S \mid \epsilon$	$\text{First}(S') = \{ \mathbf{e}, \epsilon \}$	$\text{Follow}(S') = \{ \mathbf{e}, \$ \}$
$E \rightarrow \mathbf{b}$	$\text{First}(E) = \{ \mathbf{b} \}$	$\text{Follow}(E) = \{ \mathbf{t} \}$

$S \rightarrow \mathbf{i} E \mathbf{t} S S'$

$S \rightarrow \mathbf{a}$

$E \rightarrow \mathbf{b}$

$\text{First}(\mathbf{i} E \mathbf{t} S S') = \{ \mathbf{i} \}$

$\text{First}(\mathbf{a}) = \{ \mathbf{a} \}$

$\text{First}(\mathbf{b}) = \{ \mathbf{b} \}$

$S' \rightarrow \mathbf{e} S$

$S \rightarrow \epsilon$

$\text{First}(\mathbf{e} S) = \{ \mathbf{e} \}$

$\text{First}(\epsilon) = \{ \epsilon \}$

$\text{Follow}(S') = \{ \mathbf{e}, \$ \}$

4.4 Top-Down Parsing

 **Example 4.17 :** The following grammar

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow + TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow * FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

	First
E	(id
E'	ϵ +
T	(id
T'	ϵ *
F	(id

	Follow
E	\$)
E'	\$)
T	+ \$)
T'	+ \$)
F	+ * \$)

4.4 Top-Down Parsing



Example 4.19 : The following grammar, which abstracts the dangling-else problem, is repeated here from Example 4.22:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Edited by Foxit Reader
Copyright(C) by Foxit
Corporation, 2005-2010
For Evaluation Only.

Non-terminal	INPUT SYMBOL					
	a	b	e	i	t	S
S	<u>$S \rightarrow a$</u>			<u>$S \rightarrow iEtSS'$</u>		
S'			<u>$S' \rightarrow \epsilon$</u> <u>$S' \rightarrow eS$</u>			<u>$S' \rightarrow \epsilon$</u>
E		<u>$E \rightarrow b$</u>				??